Statistical Computing

Lab Notes

Table of Contents

Statistical Computing Lab Notes **3**
R Fundamentals and Data Handling . . . 3
Probability Theory and Logic . . . 3
Random Variables and Probability Distributions . . . 10
Z-Scores and Normal Probabilities . . . 12
Sampling Distributions and The Central Limit Theorem . . . 12
Confidence Intervals . . . 13
Hypothesis Testing Framework: Step-by-Step Guide . . . 14
Enumerative Data Analysis . . . 14
Maximum Likelihood Estimation (MLE) . . . 15
Numerical Optimization and Calculus in R . . . 16
Generalised Linear Models . . . 17
Count Data and Advanced Regression . . . 18
Density Estimation . . . 19
Nonparametric Classification and Density . . . 20
Gaussian Mixture Models (GMM) . . . 21
Mixture Models and Advanced Regression . . . 22

Statistical Computing Lab Notes

R Fundamentals and Data Handling

R is a language for expressing statistical algorithms, likelihood-based inference, and simulation studies rather than solely a data analysis tool.

Reading and Creating Data

- **Import CSV:** `data <- read.csv("file.csv")` — loads a file into a data frame.
- **Create a vector:** `x <- c(1, 2, 3, 4, 5)`
- **Create a data frame:** `df <- data.frame(x = c(1,2,3), y = c(4,5,6))`
- **View structure:** `str(df)`, `head(df)`, `summary(df)`
- **Access a column:** `df$column_name` or `df[["column_name"]]`
- **Filter rows:** `df[df$x > 2, ]`

Core R Functions

- **Reproducibility:** Control random number generation to ensure simulations can be accurately duplicated by using `set.seed()`.
- **Vectorized Operations:** Most R functions operate on vectors element-wise. Logical indexing is essential for subsetting data based on conditions (e.g., `vec[vec > 0]`).
- **Missing Data:** Real datasets frequently contain `NA` values. Aggregate functions require explicit handling using the `na.rm = TRUE` argument to avoid errors.
- **Apply Family:** Avoid explicit loops for iterative operations. Apply functions across subsets or groups using functions like `sapply()`, `tapply()`, and `aggregate()`.
- **Numerical Stability:** How quantities are computed matters numerically; an iteratively stable algorithm prevents precision loss when calculating statistics for datasets with extreme values.
- **Numerically Stable Mean:**

$$\text{mean} = \frac{\sum(x_i - x_1) + n \cdot x_1}{n}$$

Probability Theory and Logic

Probability quantifies uncertainty and forms the backbone of statistical simulations and models.

- **Sample Space:** The complete set of all possible outcomes, mathematically denoted as Ω.
- **Mutually Exclusive Events:** Events that cannot occur at the same time, meaning $P(A \cap B) = 0$.
- **Independent Events:** Two events are independent if the occurrence of one does not affect the probability of the other.
- **Independence Verification:**

$$P(A \cap B) = P(A) \cdot P(B)$$

- **General Addition Rule (Union):**

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

- **Conditional Probability:**

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

- **Law of Total Probability:**

$$P(T) = P(T \mid D) \cdot P(D) + P(T \mid D^c) \cdot P(D^c)$$

- **Bayes Theorem:**

$$P(D \mid T) = \frac{P(T \mid D) \cdot P(D)}{P(T)}$$

Probability Calculations in R

In R, probabilities are derived by defining a **sample space** and **events as subsets of that space**, then counting: $P(A) = \frac{|A|}{|\Omega|}$. R has a full suite of set operations for this.

Step 1 — Define the Sample Space and Events as Vectors

```
# Sample space: all possible outcomes
omega <- 1:6   # rolling a fair die

# Define events as subsets (conditions on omega)
A <- omega[omega %% 2 == 0]   # even numbers: {2, 4, 6}
B <- omega[omega > 4]          # greater than 4: {5, 6}

# Or define explicitly
A <- c(2, 4, 6)
B <- c(5, 6)
```

Step 2 — R Set Operations

Operation	Math	R function	Result (A, B above)
Intersection	$A \cap B$	`intersect(A, B)`	`{6}`
Union	$A \cup B$	`union(A, B)`	`{2, 4, 5, 6}`
Complement	A^c	`setdiff(omega, A)`	`{1, 3, 5}`
Difference	$A \setminus B$	`setdiff(A, B)`	`{2, 4}`
Membership	$x \in A$	`x %in% A`	`TRUE/FALSE`

Step 3 — Calculate Probabilities from Sets

```
omega <- 1:6
A <- c(2, 4, 6)   # even
B <- c(5, 6)       # > 4

# Basic probabilities
P <- function(event, space = omega) length(event) / length(space)

P(A)              # P(A) = 3/6 = 0.5
P(B)              # P(B) = 2/6 = 0.333

# Complement: P(A^c)
P(setdiff(omega, A))          # 0.5

# Intersection (AND): P(A & B)
P(intersect(A, B))            # P({6}) = 1/6 = 0.167

# Union (OR): P(A | B)
P(union(A, B))                # P({2,4,5,6}) = 4/6 = 0.667

# Conditional: P(B | A) = P(A & B) / P(A)
P(intersect(A, B)) / P(A)     # 0.333

# Independence check: P(A & B) == P(A) * P(B)?
isTRUE(all.equal(P(intersect(A, B)), P(A) * P(B)))   # FALSE -> dependent
```

Working from a Data Frame (Real Dataset) When your sample space is rows in a data frame, use logical indexing and `nrow()`.

```
# Example: 200 students, columns: passed (TRUE/FALSE), studied (TRUE/FALSE)
df <- data.frame(
  passed  = c(rep(TRUE, 120), rep(FALSE, 80)),
  studied = c(rep(TRUE, 100), rep(FALSE, 20), rep(TRUE, 30), rep(FALSE, 50))
)

n <- nrow(df)   # size of sample space

# Define events as logical vectors (one TRUE/FALSE per row)
A <- df$passed          # event A: student passed
B <- df$studied         # event B: student studied

# Probabilities
P_A <- mean(A)          # P(passed)        - mean() of logical = proportion
P_B <- mean(B)          # P(studied)

# Intersection: P(A & B) -- passed AND studied
```

```r
P_AB <- mean(A & B)

# Union: P(A | B) -- passed OR studied (or both)
P_AuB <- mean(A | B)

# Complement: P(A^c) - did NOT pass
P_Ac <- mean(!A)

# Conditional: P(A | B) - P(passed | studied)
P_A_given_B <- mean(A[B])   # subset A to rows where B is TRUE, then average

# Independence check
isTRUE(all.equal(P_AB, P_A * P_B))
```

Key insight: for logical vectors, `mean()` gives proportions, `&` is intersection, `|` is union, `!` is complement, and `[B]` subsets to condition on B.

Working from a Frequency Table

```r
# Contingency table - rows: Defective/OK, cols: Machine A / Machine B
tbl <- matrix(c(10, 40, 5, 45), nrow = 2,
              dimnames = list(c("Defective", "OK"), c("MachineA", "MachineB")))

n <- sum(tbl)

# Marginal probabilities
P_defective  <- sum(tbl["Defective", ]) / n   # P(Defective)
P_machineA   <- sum(tbl[, "MachineA"]) / n    # P(Machine A)

# Joint probability: P(Defective & Machine A)
P_def_and_A  <- tbl["Defective", "MachineA"] / n

# Conditional: P(Defective | Machine A)
P_def_given_A <- tbl["Defective", "MachineA"] / sum(tbl[, "MachineA"])

# Or equivalently:
P_def_given_A <- P_def_and_A / P_machineA

# Independence check
isTRUE(all.equal(P_def_and_A, P_defective * P_machineA))
```

Complement: $P(A^c) = 1 - P(A)$ "The probability that A does NOT happen."

```r
omega <- 1:6
A <- c(2, 4, 6)
```

```r
P_not_A <- length(setdiff(omega, A)) / length(omega)    # 0.5
# or equivalently
P_not_A <- 1 - length(A) / length(omega)
```

Use when: you want "at least one", "none", or "not A" style questions.

Union (OR): $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ "The probability that A or B (or both) happen."

```r
omega <- 1:6
A <- c(2, 4, 6)   # even
B <- c(5, 6)       # > 4
P <- function(event) length(event) / length(omega)

P(union(A, B))                          # direct: 4/6 = 0.667
P(A) + P(B) - P(intersect(A, B))        # formula: same result
```

Special case — **mutually exclusive** events ($A \cap B = \emptyset$):

```r
A <- c(1, 3)   # odd < 4
B <- c(5)      # odd > 4
# No overlap, so:
length(intersect(A, B)) == 0   # TRUE - mutually exclusive
P(union(A, B))                  # = P(A) + P(B) = 0.5
```

Intersection (AND): $P(A \cap B)$ "The probability that both A and B happen."

```r
omega <- 1:6
A <- c(2, 4, 6)   # even
B <- c(5, 6)       # > 4
P <- function(event) length(event) / length(omega)

# Direct from sets:
P(intersect(A, B))   # P({6}) = 1/6

# If A and B are INDEPENDENT, multiply (verify first):
P(A) * P(B)          # only valid when independence holds

# General multiplication rule:
# P(A & B) = P(B|A) * P(A)
P_B_given_A <- length(intersect(A, B)) / length(A)
P_B_given_A * P(A)   # = 1/6
```

Conditional Probability: $P(B \mid A) = \frac{P(A \cap B)}{P(A)}$ "The probability of B, given that A has already occurred."

```
omega <- 1:6
A <- c(2, 4, 6)   # even
B <- c(5, 6)       # > 4
P <- function(event) length(event) / length(omega)

# Formula approach:
P(intersect(A, B)) / P(A)   # 0.333

# Set approach - restrict sample space to A, then ask what fraction is in B:
restricted <- A   # new sample space is A
mean(restricted %in% B)      # 0.333 - same result
```

Use when: "given that ...", "knowing that ...", "if A occurred ..."

Independence Check: $P(A \cap B) = P(A) \cdot P(B)$ "Are A and B independent?"

```
omega <- 1:6
A <- c(2, 4, 6)
B <- c(5, 6)
P <- function(event) length(event) / length(omega)

P(intersect(A, B))          # 1/6 = 0.1667
P(A) * P(B)                 # 0.5 * 0.333 = 0.1667

isTRUE(all.equal(P(intersect(A, B)), P(A) * P(B)))   # TRUE -> independent
```

Law of Total Probability "The overall probability of B, broken down across a partition of the sample space."

$$P(B) = \sum_i P(B \mid A_i) \cdot P(A_i)$$

```
# Example: 600 items from 3 machines
# Each machine produces a different share, with different defect rates
items <- data.frame(
  machine  = c(rep("A", 200), rep("B", 250), rep("C", 150)),
  defective = c(rep(c(TRUE, FALSE), c(20, 180)),    # Machine A: 10% defective
                rep(c(TRUE, FALSE), c(12, 238)),    # Machine B: ~5% defective
                rep(c(TRUE, FALSE), c(30, 120)))    # Machine C: 20% defective
)

# Partition: the three machines
```

```r
partition <- split(items, items$machine)

# P(B | A_i): defect rate within each machine group
P_B_given_Ai <- sapply(partition, function(g) mean(g$defective))

# P(A_i): proportion of items from each machine
P_Ai <- sapply(partition, nrow) / nrow(items)

# Law of Total Probability
P_B <- sum(P_B_given_Ai * P_Ai)

# Verify directly:
mean(items$defective)    # should match P_B
```

Use when: problem has multiple groups/scenarios and you need the overall probability.

Bayes' Theorem: $P(A \mid B) = \frac{P(B|A) \cdot P(A)}{P(B)}$ "Reverse a conditional probability — update your belief after observing B."

```r
# Same items data frame from above
# Question: given an item is defective, what is P(it came from Machine C)?

B  <- items$defective                    # event B: defective
Ac <- items$machine == "C"               # event A: from Machine C

# All probabilities derived from the data:
P_B       <- mean(B)                     # P(defective)
P_Ac      <- mean(Ac)                    # P(Machine C)
P_B_Ac    <- mean(B & Ac)                # P(defective AND Machine C)

# Bayes:
P_Ac_given_B <- P_B_Ac / P_B             # P(Machine C | defective)

# Verify directly (restrict to defective rows):
mean(items$machine[items$defective] == "C")    # same answer
```

Quick Reference: Which Formula to Use?

Question type	Math	R (from sets)
Probability of A	$\|A\|/\|\Omega\|$	`length(A) / length(omega)`
A does NOT happen	$1 - P(A)$	`length(setdiff(omega, A)) / length(omega)`
A or B	$P(A) + P(B) - P(A \cap B)$	`P(union(A, B))`

Question type	Math	R (from sets)
A or B, mutually exclusive	$P(A) + P(B)$	`P(union(A, B))` — intersection is empty
A and B	$P(A \cap B)$	`P(intersect(A, B))`
A and B, independent	$P(A) \cdot P(B)$	`P(A) * P(B)` — only after confirming independence
B given A	$P(A \cap B)/P(A)$	`P(intersect(A,B)) / P(A)` or `mean(B_lgl[A_lgl])`
Are A and B independent?	$P(A \cap B) \stackrel{?}{=} P(A)P(B)$	`isTRUE(all.equal(P(intersect(A,B)), P(A)*P(B)))`
Overall P across groups	$\sum P(B \mid A_i)P(A_i)$	`sum(P_B_giv * P_Ai)` or `mean(B_col)` directly
Reverse conditional	Bayes' Theorem	`P_AB / P_B` or `mean(A_col[B_col])`

Random Variables and Probability Distributions

A random variable is a numerical mapping of outcomes from a sample space. In R, distributions are accessed using specific prefix letters: `d` (probability density/mass function), `p` (cumulative distribution function), `q` (quantile function), and `r` (random deviate generation).

- **Expected Value:** The long-run average of a distribution over many trials, denoted as $E(X)$ or μ.
- **Variance:** The measure of spread or dispersion around the expected value, denoted as $\text{Var}(X)$ or σ^2.

Discrete Distributions

- **Binomial Distribution:** Models the number of successes in n independent trials with a constant probability of success p. Use `dbinom(x, size, prob)`.
- **Poisson Distribution:** Models the count of rare events occurring within a fixed interval, where the mean rate is λ. The variance is equal to the expected value. Use `dpois(x, lambda)`.
- **Hypergeometric Distribution:** Models drawing from a finite population *without* replacement, making the events dependent. Use `dhyper(x, m, n, k)`.
- **Negative Binomial Distribution:** Models the number of failures before a specified number of successes occurs. Use `dnbinom(x, size, prob)`.
- **Expected Value (Binomial):**

$$E(X) = n \cdot p$$

- **Variance (Binomial):**

$$\text{Var}(X) = n \cdot p \cdot (1 - p)$$

- **Poisson Approximation to Binomial:** If a Binomial distribution has a large number of trials ($n \geq 20$) and a small probability of success ($p \leq 0.05$), the Poisson distribution acts as an excellent approximation using:

$$\lambda = n \cdot p$$

Calculating Probabilities in R

Every distribution in R follows a four-function naming convention: prefix + distribution name.

Prefix	Function	Returns
`d`	Density / PMF	$P(X = x)$ — exact probability (discrete) or density (continuous)
`p`	CDF	$P(X \leq x)$ — cumulative probability up to x
`q`	Quantile	The value x such that $P(X \leq x) = p$
`r`	Random	Generates random samples from the distribution

Binomial Examples

```r
# P(X = 3) where X ~ Bin(10, 0.4)
dbinom(3, size = 10, prob = 0.4)

# P(X <= 3)
pbinom(3, size = 10, prob = 0.4)

# P(X >= 4) = 1 - P(X <= 3)
1 - pbinom(3, size = 10, prob = 0.4)

# P(2 <= X <= 5)
pbinom(5, 10, 0.4) - pbinom(1, 10, 0.4)
```

Poisson Examples

```r
# P(X = 2) where X ~ Pois(lambda = 3)
dpois(2, lambda = 3)

# P(X <= 2)
ppois(2, lambda = 3)

# P(X > 2)
1 - ppois(2, lambda = 3)
```

Normal Examples

```r
# P(X < 70) where X ~ N(65, 9)  [sd = 3]
pnorm(70, mean = 65, sd = 3)

# P(X > 70)
pnorm(70, mean = 65, sd = 3, lower.tail = FALSE)

# P(62 < X < 68)
pnorm(68, 65, 3) - pnorm(62, 65, 3)

# Find x such that P(X <= x) = 0.95
qnorm(0.95, mean = 65, sd = 3)
```

Continuous Distributions

- **Uniform Distribution:** Models outcomes that are equally likely across a continuous interval. Use `dunif(x, min, max)`.
- **Normal Distribution:** A symmetric, bell-shaped distribution completely defined by its mean μ and standard deviation σ. Use `dnorm(x, mean, sd)`.
- **Gamma Distribution:** Models positive, right-skewed data such as waiting times or insurance claims. It is defined by a shape parameter α and rate parameter β. Use `dgamma(x, shape, rate)`.
- **Exponential Distribution:** A specific case of the Gamma distribution modelling waiting times between events in a Poisson process. Use `dexp(x, rate)`.
- **Chi-Squared Distribution:** A right-skewed distribution dependent on degrees of freedom, often used in goodness-of-fit and likelihood ratio tests. Use `dchisq(x, df)`.

Z-Scores and Normal Probabilities

A Z-score standardizes a data point to represent how many standard deviations it falls away from the mean, allowing for the use of standard normal tables or probability functions to find the area under the curve.

- **Z-Score Formula:**

$$Z = \frac{X - \mu}{\sigma}$$

To calculate specific continuous probabilities in R using the Normal Distribution:

1. **Probability Less Than ($P(X < x)$):** Use `pnorm(x, mean, sd)`.
2. **Probability Greater Than ($P(X > x)$):** Use `pnorm(x, mean, sd, lower.tail = FALSE)` or `1 - pnorm(x, mean, sd)`.
3. **Probability Between ($P(a < X < b)$):** Calculate the cumulative area up to b and subtract the cumulative area up to a using `pnorm(b, mean, sd) - pnorm(a, mean, sd)`.

Sampling Distributions and The Central Limit Theorem

The behavior of sample statistics over repeated sampling forms the basis of frequentist inference.

- **Central Limit Theorem:** Regardless of the underlying population distribution, the sampling distribution of the sample mean $\bar{X}$ approaches a Normal distribution as the sample size n increases.
- **Standard Error:** The standard deviation of the sampling distribution, quantifying the precision of the sample mean.
- **Standard Error Formula:**

$$SE = \frac{\sigma}{\sqrt{n}}$$

Confidence Intervals

A confidence interval gives a range of plausible values for a population parameter based on sample data.

- **Interpretation:** If we repeated the sampling procedure many times, $(1-\alpha)\%$ of the constructed intervals would contain the true parameter.

CI for a Mean (Known σ or Large n, use Z)

$$\bar{x} \pm z_{\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}$$

```
# Manual Z-based CI
z <- qnorm(0.975)          # 1.96 for 95% CI
lower <- xbar - z * (sigma / sqrt(n))
upper <- xbar + z * (sigma / sqrt(n))
```

CI for a Mean (Unknown σ, use t)

$$\bar{x} \pm t_{\alpha/2,\, n-1} \cdot \frac{s}{\sqrt{n}}$$

```
# R computes this automatically
t.test(x, conf.level = 0.95)$conf.int
```

CI for a Proportion

```
prop.test(x, n, conf.level = 0.95)$conf.int
# or for exact binomial CI:
binom.test(x, n, conf.level = 0.95)$conf.int
```

Key Values

Confidence Level	$z_{\alpha/2}$
90%	1.645
95%	1.960
99%	2.576

- **Decision rule:** If μ_0 (the hypothesised value) lies **outside** the CI, reject H_0.

Hypothesis Testing Framework: Step-by-Step Guide

Hypothesis testing is a structured way to evaluate competing claims about population parameters based on sample data.

1. **Define Hypotheses:** Establish the Null Hypothesis (H_0), representing the status quo or "no difference", and the Alternative Hypothesis (H_1), representing the specific effect you are testing for (e.g., $H_1 : \mu_1 \neq \mu_2$ for a two-sided test or $H_1 : \mu_1 < \mu_2$ for a one-sided test). Define your significance level α (typically 0.05).
2. **Check for Normality:** Run the Shapiro-Wilk test via `shapiro.test(x)` on your data.
   - If $p > 0.05$: The data does not significantly deviate from a Normal distribution. Proceed with Parametric tests.
   - If $p < 0.05$: The data is not normal. Proceed with Non-Parametric tests.
3. **Check for Equal Variance (Parametric Only):** If comparing two independent normally distributed groups, use Bartlett's test via `bartlett.test(list(group1, group2))`.
   - If $p > 0.05$: Assume equal variances.
   - If $p < 0.05$: Variances are unequal.
4. **Select and Execute the Correct Test:**
   - *Normal + Equal Variance:* Standard Two-Sample T-Test `t.test(x, y, alternative="two.sided")`.
   - *Normal + Unequal Variance:* Welch's T-Test (R handles this automatically in `t.test` by default).
   - *Non-Normal (Independent):* Wilcoxon Rank Sum Test `wilcox.test(x, y)`.
   - *Paired Data (Before/After):* Add the `paired = TRUE` argument to `t.test` or `wilcox.test`.
   - *Categorical/Proportions:* Use `binom.test()` for exact binomial tests on count data or `prop.test()` for proportions.
5. **Interpret the Results:** Look at the p-value in the R output.
   - If $p < \alpha$: Reject the Null Hypothesis (H_0) and accept the Alternative Hypothesis (H_1).
   - If $p > \alpha$: Fail to reject the Null Hypothesis (H_0).

Enumerative Data Analysis

Analysis of qualitative, categorical data relies on comparing observed frequencies against expected frequencies.

- **Goodness-of-Fit Test:** Evaluates if a single categorical variable matches a claimed theoretical distribution. Use `chisq.test(x, p = expected_probs)`.
- **Test of Independence:** Evaluates if two categorical variables are associated or independent. Use `chisq.test(table_data)`.
- **Rule of 5 Assumption:** Chi-squared tests require at least 80% of expected counts to be ≥ 5, and no expected count to be < 1.
- **Fisher's Exact Test:** The non-parametric alternative for independence testing when the Rule of 5 assumption is violated in small samples. Use `fisher.test(table_data)`.
- **Effect Size:** Quantifies the magnitude of the difference (e.g., Cohen's d for means, Phi coefficient for categorical associations) independent of sample size.
- **Expected Counts Formula:**

$$E_{ij} = \frac{(\text{Row Total})(\text{Column Total})}{\text{Grand Total}}$$

- **Chi-Squared Test Statistic:**

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

- **Phi Coefficient Formula:**

$$\phi = \sqrt{\frac{\chi^2}{n}}$$

- **Cohen's d Formula:**

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s}$$

Creating Contingency Tables in R

```r
# From raw data
tbl <- table(df$var1, df$var2)

# From counts directly
tbl <- matrix(c(10, 20, 30, 40), nrow = 2, byrow = TRUE)

# Run chi-squared test
chisq.test(tbl)

# Fisher's exact test (small samples)
fisher.test(tbl)
```

Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation is a method for estimating the parameters of a statistical model by selecting the values that make the observed data most probable.

- **Log-Likelihood:** Because multiplying many small probabilities causes numerical instability (underflow), it is standard practice to take the natural logarithm. R density functions natively support this via the `log = TRUE` argument.
- **The MLE Concept:** The Maximum Likelihood Estimate (MLE) is the specific parameter value $\hat{\theta}$ that maximizes the log-likelihood function.
- **Analytical Solutions:** Some models have closed-form MLEs. For a Poisson distribution, $\hat{\lambda}$ is the sample mean $\bar{x}$. For a Binomial distribution, $\hat{p}$ is the sample success rate $\bar{y}/n$.
- **Likelihood Surfaces:** When a model contains two unknown parameters (e.g., Normal distribution with unknown μ and σ), the log-likelihood becomes a 3D surface over a parameter space.
- **Likelihood Function:**

$$L(\theta) = \prod_{i=1}^{n} f(x_i \mid \theta)$$

- **Log-Likelihood Function:**

$$\ell(\theta) = \sum_{i=1}^{n} \log f(x_i \mid \theta)$$

- **Likelihood Ratio Test Statistic:**

$$\Lambda = -2\left[\ell(\hat{\theta}_0) - \ell(\hat{\theta})\right]$$

Numerical Optimization and Calculus in R

When analytical differentiation yields no closed-form solution (such as the shape parameter α in a Gamma distribution), numerical optimization algorithms must be used to find the MLE.

- **Minimization via `mle()`:** R's `mle()` function (from the `stats4` package) estimates parameters by *minimizing* a provided negative log-likelihood function. It requires the negative log-likelihood function, a `start` list of initial guesses, and `nobs` (number of observations).
- **Checking Convergence:** An optimization model that hasn't converged can look perfectly reasonable but be completely wrong. Always check `fit@details$convergence`. A code of 0 indicates success, while non-zero values (1, 52, 54) indicate failures or numerical faults.
- **Symbolic Differentiation:** R can compute analytical derivatives using the `D(expression, variable)` function.

Optimization Methods

Under the hood, `mle()` utilizes the general-purpose `optim()` function, which relies on different algorithms to navigate the parameter space.

- **Nelder-Mead:** A derivative-free method that navigates using a simplex (geometric shape). It is robust and handles irregular surfaces well, but can be slow to converge. It is the default method in `optim()`.
- **BFGS:** A quasi-Newton method that approximates the Hessian matrix (second derivatives) from successive gradient evaluations. It is much faster than Nelder-Mead on smooth log-likelihoods and is the default algorithm in `mle()`.
- **Gradient Ascent:** An iterative algorithm that steps in the direction of the gradient by a fixed step size γ (the learning rate). It is sensitive to γ and requires manual implementation using analytical derivatives.
- **L-BFGS-B and Brent:** Specialized algorithms employed when the parameter space must be constrained by lower and upper bounds (e.g., standard deviation and probabilities cannot be negative). `Brent` is strictly for single-parameter bounded optimization.
- **Gradient Ascent Update Rule:**

$$\theta^{(t+1)} = \theta^{(t)} + \gamma \cdot \left.\frac{\partial \ell}{\partial \theta}\right|_{\theta^{(t)}}$$

Generalised Linear Models

Generalised Linear Models (GLMs) extend standard linear regression to allow for response variables that have error distribution models other than a normal distribution (e.g., binomial for logistic regression, Poisson for count data).

Linear Models

- **Standard Linear Regression:** Fit a linear model using `lm(y ~ x)`.
- **Zero-Intercept Model:** Suppress the intercept by using `- 1` or `0 +` in the formula. This is used when theory dictates the line must pass through the origin (e.g., Hubble's Law $v = H_0 D$).
- **High Leverage:** An observation is considered to have high leverage if it has extreme values for predictor variables compared to the rest of the dataset.
- **Check Leverage in R:**

```
hatvalues(model)
```

- **Nested Models & Likelihood Ratio Test (LRT):** Two models are nested if one restricts the parameters of the other (e.g., M_0 fixes the slope to a specific value, while M_1 estimates it freely). Compare them to see if the extra parameters significantly improve the fit.

```
# M0: y = B0 + 0.02*x (slope restricted to 0.02)
m0 <- lm(BAL ~ 1 + offset(0.02 * Beers))

# M1: y = B0 + B1*x (slope freely estimated)
m1 <- lm(BAL ~ Beers)

# Compare models using LRT
anova(m0, m1, test = "LRT")
# If significant, reject the restricted hypothesis
```

Logistic Regression

Models the probability of a binary outcome.

- **Model Fitting:** Use `glm()` with the `family = binomial` argument.
- **Reference Categories:** Pay attention to how categorical variables are encoded; R automatically treats the first level alphabetically as the reference category (coded 0).
- **Odds-Ratios:** Exponentiating the model coefficients translates them into odds ratios.
- **Fitting and Odds-Ratios in R:**

```
# Fit a logistic regression model
model <- glm(outcome ~ predictor1, family = binomial, data = df)
```

```r
# Calculate Odds Ratios
exp(coef(model))
```

Odds-Ratio Interpretation: A common misinterpretation is stating that an Odds-Ratio of 4 means an event is 4 times more likely to happen in terms of probability. It means the *odds* $(p/(1-p))$ change by a factor of 4, and the impact on the absolute probability depends heavily on the baseline risk.

- **Best Model Selection:** Evaluate if dropping predictors with high p-values results in a better model. Compare models using multiple criteria to ensure they agree:

1. **LRT:** `anova(model_full, model_reduced, test = "LRT")`
2. **Akaike Information Criterion:** `AIC(model)`
3. **Bayesian Information Criterion:** `BIC(model)`

Poisson Regression

Models count data, such as the number of claims or events.

- **Model Fitting:** Use `glm()` with the `family = poisson` argument.
- **Offsets:** When modeling totals across groups of different sizes (e.g., predicting claim counts without accounting for the number of policyholders), the model must be adjusted to predict the *rate*. Include an `offset()` term of the log of the exposure variable.
- **Fitting with an Offset in R:**

```r
# Predict number of claims, offsetting by the number of policyholders
# This effectively models the rate of claims per holder
poisson_model <- glm(Claims ~ District + Group + Age + offset(log(Holders)),
                     family = poisson, data = Insurance)
```

- **Categorical Contrasts:** R may use orthogonal polynomials for numeric factors, showing linear/quadratic relationships as categories increase. To interpret coefficients relative to a baseline category instead, force standard dummy variables:

```r
# Force dummy variable treatment for a factor
Insurance$Age <- factor(Insurance$Age)
contrasts(Insurance$Age) <- contr.treatment(levels(Insurance$Age))
```

Count Data and Advanced Regression

When modeling counts or strictly positive continuous variables, standard linear regression is often inappropriate due to variance and bounds issues.

Poisson and Negative Binomial Regression

Models count data (e.g., number of insurance claims or bike rentals). The standard Poisson model assumes the variance is equal to the mean.

- **Poisson Model Fitting:** Fit using `glm()` with `family = poisson(link = "log")`.

- **Offsets:** If predicting counts over varying exposures (like different numbers of policyholders), use an `offset()` to model the *rate* rather than the raw count.
- **Overdispersion:** If the variance of the data significantly exceeds the mean, the Poisson model assumptions are violated.
- **Negative Binomial Model:** An extension of the Poisson model that introduces an extra parameter to account for overdispersion.
- **Fitting Count Models in R:**

```
library(MASS)

# Poisson model with an offset for exposure
poisson_model <- glm(Claims ~ District + Group + Age + offset(log(Holders)),
                     family = poisson(link = "log"), data = Insurance)

# Negative Binomial model for overdispersed data
nb_model <- glm.nb(Claims ~ District + Group + Age + offset(log(Holders)),
                   data = Insurance)

# Compare improvements in log-likelihood and deviance
summary(nb_model)
```

Gamma Regression

Gamma regression is useful for modeling continuous, strictly positive data that is typically right-skewed.

- **Model Fitting:** Use `glm()` with `family = Gamma()` (often with a log link depending on the relationship).

Density Estimation

Kernel density estimation (KDE) is a non-parametric way to estimate the probability density function of a random variable, effectively creating a smoothed version of a histogram.

- **Kernels:** The shape of the "bump" placed at each data point. Common choices include Rectangular and Epanechnikov (the standard default in R is often Gaussian).
- **Bandwidth:** Controls the smoothness of the estimate. A larger bandwidth creates a smoother curve (potentially underfitting), while a smaller bandwidth creates a more jagged curve (potentially overfitting).
- **Bandwidth Selection Methods:**
  - **Rule-of-Thumb:** A standard heuristic computed using `bw.nrd0`.
  - **Cross-Validation:** A data-driven approach computed using `bw = "ucv"` (unbiased cross-validation), which frequently selects a smaller bandwidth than the rule-of-thumb.
- **Sum of Normals Concept:** The standard density estimate provided by `density()` is mathematically equivalent to placing an individual Normal distribution curve centered on

every single data point—where the standard deviation of each normal curve equals the chosen bandwidth—and summing them all together.

- **Creating Density Estimates in R:**

```r
# Standard density estimate
turtle_density <- density(Turtles)
plot(turtle_density)

# Specifying an exact bandwidth and a rectangular kernel
plot(density(Turtles, bw = 1, kernel = "rectangular"))

# Specifying an Epanechnikov kernel with a bandwidth of 5
plot(density(Turtles, bw = 5, kernel = "epanechnikov"))

# Using unbiased cross-validation to automatically determine bandwidth
plot(density(Turtles, bw = "ucv", kernel = "epanechnikov"))

# Extracting the calculated rule-of-thumb bandwidth from a density object
h <- turtle_density$bw
```

Nonparametric Classification and Density

Kernel Density Estimation (KDE) can be used to build a non-parametric classifier. By estimating the conditional densities of a predictor for each class, you can apply Bayes' Theorem to find the posterior probability of a class belonging to a certain category given a predictor value.

- **Posterior Probability Formula:**

$$P(C = 1 \mid X = x) = \frac{P(X = x \mid C = 1) \cdot P(C = 1)}{P(X = x)}$$

Alternatively, using the expanded denominator (Law of Total Probability):

```
$$P(C=1 \mid X=x) = \frac{P(X=x \mid C=1) \cdot P(C=1)}{P(X=x \mid C=1) \cdot P(C=1) + P(X=x \
```

- **Implementation in R:**

```r
# Define overall density and class-conditional densities
f <- density(df$predictor)
f0 <- density(df$predictor[df$class == 0]) # Density for Class 0
f1 <- density(df$predictor[df$class == 1]) # Density for Class 1

# Calculate the prior probability of Class 1
p <- mean(df$class == 1)

# Calculate Posterior Probability: P(Class=1 | predictor)
# f1$y is the density of class 1, f$y is the overall density
prob_1 <- (f1$y * p) / f$y

# Plot the probability of being in Class 1 across predictor values
```

```r
plot(f$x, prob_1, type = "l", xlab = "Predictor", ylab = "P(Class 1 | Predictor)")
abline(h = 0.5, lty = 3) # Add a 50% decision boundary line
```

2D Density Estimation

KDE can be extended to two dimensions to visualize clusters or relationships between two continuous variables.

```r
library(MASS)

# Compute 2D kernel density estimate
dens <- kde2d(x, y)

# Visualise the 2D density
contour(dens) # Contour plot representation
image(dens)   # Heatmap/color representation
```

Gaussian Mixture Models (GMM)

When data exhibits multiple modes (peaks), a single standard distribution is insufficient. A mixture model represents the data as a combination of multiple distributions (usually Normal distributions).

- **Parameters:** A k-component Normal mixture model has $3k - 1$ free parameters:
  - k means (μ)
  - k standard deviations (σ)
  - $k - 1$ mixing weights (λ), because the weights must sum exactly to 1.
- **Expectation-Maximization (EM):** The `mixtools` library uses the EM algorithm to iteratively estimate the parameters of the mixture components.
- **Fitting a Mixture Model in R:**

```r
library(mixtools)

# Fit models with varying numbers of components (k)
mix2 <- normalmixEM(data, k = 2)
mix3 <- normalmixEM(data, k = 3)

# Plot the fitted density curves over the data
plot(mix2, whichplots = 2)
```

Model Selection (AIC & BIC)

To determine the optimal number of components (k), compare the models using the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC), which penalize models for adding unnecessary complexity (extra parameters). The model with the lowest AIC/BIC is generally preferred.

```r
n <- length(data)

# Calculate parameters (3k - 1)
```

```r
p2 <- 3 * 2 - 1  # 5 parameters for k=2
p3 <- 3 * 3 - 1  # 8 parameters for k=3

# Manual AIC calculation: 2 * (-logLikelihood) + 2 * parameters
# Note: mixtools returns positive loglik, so use +2*mix$loglik or standard formula logic depen
AIC2 <- -2 * mix2$loglik + 2 * p2

# Manual BIC calculation: 2 * (-logLikelihood) + log(n) * parameters
BIC2 <- -2 * mix2$loglik + log(n) * p2
```

Maximum Likelihood Estimation for Mixtures

You can also manually fit a mixture model by writing a negative log-likelihood function and optimizing it. Because variance and probabilities have strict bounds, you must use a bounded optimization algorithm like `"L-BFGS-B"`.

```r
library(stats4)

# Negative log-likelihood for a 2-component Normal mixture
negloglik <- function(lambda, mu1, s1, mu2, s2) {
  -sum(log(lambda * dnorm(data, mu1, s1) + (1 - lambda) * dnorm(data, mu2, s2)))
}

# Minimize negative log-likelihood using mle()
fit <- mle(negloglik,
           start = list(lambda = 0.5, mu1 = 5, s1 = 5, mu2 = 25, s2 = 8),
           method = "L-BFGS-B",
           lower = c(0.01, -Inf, 0.01, -Inf, 0.01), # Bounds to prevent negative variance
           upper = c(0.99, Inf, Inf, Inf, Inf))

summary(fit)
```

Mixture Models and Advanced Regression

Mixture of Regressions

Sometimes a dataset contains multiple subgroups with entirely different linear relationships, but the group memberships of the data points are unknown. A regression mixture model estimates these separate regression lines simultaneously.

- **Interaction Models:** If the groups *are* known, an interaction term in a standard linear model can fit the separate lines (e.g., `lm(y ~ group * x)`).
- **Regression Mixtures:** If the groups are *unknown*, the model estimates the lines and assigns a posterior probability to each observation indicating which line it likely belongs to.
- **Fitting a Regression Mixture in R:**

```r
library(mixtools)

# Known groups: Interaction model
```

```r
fit_interaction <- lm(Gas ~ Insul * Temp, data = whiteside)

# Unknown groups: Regression mixture
# arbvar = FALSE constrains both components to share the same variance
fit_mix <- regmixEM(whiteside$Gas, whiteside$Temp, arbvar = FALSE, k = 2)

# Extracting regression coefficients for the mixture components
beta <- fit_mix$beta

# Extracting posterior probabilities of group membership
posterior_probs <- fit_mix$posterior
assigned_group <- ifelse(posterior_probs[, 1] > 0.5, "Component 1", "Component 2")
```

Multivariate Mixture Models

Gaussian Mixture Models (GMMs) can be extended to multiple dimensions to cluster complex multivariate data, such as flow cytometry measurements where distinct cell populations overlap.

- **Bivariate Normal Mixtures:** Use `mvnormalmixEM()` to fit mixtures on 2D data.
- **Model Parameters:** For a k-component mixture of bivariate Normal distributions, the total number of parameters is $p = 6k - 1$. Each component contributes:
  - 2 means (μ_x, μ_y)
  - 3 unique covariance parameters $(\sigma_x^2, \sigma_y^2, \text{cov}_{xy})$
  - And there are $k - 1$ free mixing weights (λ) because they must sum to 1.
- **Fitting a Bivariate Mixture in R:**

```r
# Fit 2-component and 3-component models
m2 <- mvnormalmixEM(data_matrix, k = 2)
m3 <- mvnormalmixEM(data_matrix, k = 3)

# Visualise the density ellipses over a scatterplot
plot(m3, whichplots = 2)
```

- **Model Selection for Bivariate Mixtures:**

```r
n <- nrow(data_matrix)

# Number of parameters for k=2 and k=3
p2 <- 6 * 2 - 1
p3 <- 6 * 3 - 1

# Calculate AIC and BIC manually for comparison
AIC2 <- -2 * m2$loglik + 2 * p2
BIC2 <- -2 * m2$loglik + log(n) * p2
```

Interpreting Gamma Regression

Gamma regression is used for strictly positive, right-skewed continuous data (e.g., rainfall, hospital wait times). With a `log` link function, the model equations look like this:

$$\log(\mu) = \beta_0 + \beta_1 x_1 + ...$$

Or equivalently:

$$\mu = e^{\beta_0 + \beta_1 x_1 + ...}$$

- **Multiplicative Effects:** Because of the log link, the coefficients must be exponentiated to be interpreted on the original scale. Instead of adding a fixed amount to the mean, an exponentiated coefficient *multiplies* the expected response.
- **Fitting and Interpreting in R:**

```
# Fit Gamma regression with a log link
fit_gamma <- glm(stay ~ age + severity + complication,
                 family = Gamma(link = "log"), data = hospital)

# Exponentiate the coefficients to get the multiplicative effects
exp(coef(fit_gamma))

# Example Interpretation:
# If exp(coef) for a "complication" factor is 1.5, developing a complication
# increases the expected length of stay by a factor of 1.5 (a 50% increase).
```